Urban Data Integration Platform

Yixin Xia, Xinyue Feng, Hsi-Jui Chang

September 20, 2026

1 Analytical Requirements

The platform must satisfy the following functional requirements to support analytical exploration, efficient repeated execution, reusable derived datasets, and evidence-based evaluation of Spark performance:

- answer mobility and environmental questions using taxi, zone, weather, and air-quality data;
- provide reproducible Spark SQL queries and reusable materialized Delta products;
- improve performance through caching, partition pruning, broadcast joins, and Adaptive Query Execution;
- compare execution time and physical plans before and after optimization;
- verify that optimization does not change query results;
- evaluate storage overhead, remaining bottlenecks, and scalability to ten cities.

Taxi demand is defined as the number of valid trips and is attributed to the pickup zone. Missing weather or air-quality values are excluded only when required by a query and are never replaced with fabricated measurements.

2 Analytical Query Design

Each query is executed using `collect()` so that the reported time includes complete Spark execution. These measurements are initial baselines for enhancement. The following optimization experiments and evaluation will take further.

2.1 Monthly Demand by Zone

Trips are joined with the zone lookup and grouped by pickup year, month, and zone. The query retains the partition columns for later partition-pruning experiments. It returned 793 rows in 101.717 seconds. In January 2024, Midtown Center had the highest observed demand with 141,740 trips. Its high initial latency makes it a priority for optimization, although part of the time may be due to cold-start overhead.

2.2 Trip Distance by Weather

Weather codes are grouped into clear, cloudy, fog, rain, snow/sleet, and storm categories. Exact weather matches are grouped to calculate trip count, mean distance, and standard deviation. The query took 22.686 seconds. Average distance was highest under clear conditions (4.293 miles) and lowest under snow or sleet (3.222 miles).

2.3 Air Quality and Demand

Trips are first aggregated to borough-hour grain to match the $PM_{2.5}$ data and avoid repeating the same measurement for every trip. Demand is compared across $PM_{2.5}$ bands, and Pearson correlation is calculated.

The band and correlation queries took 11.322 and 8.578 seconds respectively. The overall correlation was 0.0233, indicating little linear relationship between $PM_{2.5}$ and hourly taxi demand. The repeated borough-hour aggregation is a candidate for caching or materialization.

2.4 Weather-Related Zone Variation

Trip counts are aggregated by zone and hour and joined with all observed weather hours. Zero-trip zone-hours are retained. Average hourly demand is then calculated for each weather category, and zones are ranked by demand range.

The query took 23.920 seconds. Upper East Side South had the largest range at 71.217 trips per hour, followed by Upper East Side North at 65.083. The zone–hour grid and multiple aggregations make this query computationally expensive.

2.5 Peak Travel Hours

Trips are grouped by weekday and hour, and `DENSE_RANK` retains tied peaks. The query took 15.166 seconds. Monday to Friday peaked at 18:00, Saturday at 19:00, and Sunday at 00:00.

2.6 Monthly Demand Trends

Monthly totals, average daily demand, and month-over-month changes are calculated using `LAG`. The query took 9.002 seconds. Demand increased from 2.93 million trips in January 2024 to 3.52 million in March.

The result also contains a small number of records from 2002, 2008, 2009, and 2023, while April 2024 is incomplete. These records distort trend percentages, so final interpretation should use complete 2024 months or apply an explicit study-period filter.

3 Analytical data products

From the `integrated_taxi_trips` Delta table, six reusable analytical data products are defined. Each has an explicit aggregation level and precomputes measures that would otherwise be calculated repeatedly from individual trips. Spark SQL produces the Delta tables. Table 1 shows each product's aggregation level, key measures and analytical use.

Every output includes `source_table`, `created_at`, `refreshed_at` and `schema_version`. Zone Hourly Demand Profile and Origin–Destination Flow are partitioned by pickup year and month for month-filtered queries; the smaller summaries remain unpartitioned to avoid small partitions. Each refresh rebuilds and overwrites the product's Delta snapshot.

Zone-hour and origin–destination summaries avoid repeated grouping of millions of individual trips. The weather product also avoids reconstructing zero-trip zone-hours, while the airport, fare/tip and distance products reuse their classification rules. These savings come at the cost of additional storage and full-refresh work; actual query-time benefits are evaluated separately in the benchmark.

Table 1: Aggregation levels, measures and uses of the analytical data products

Product	Aggregation level	Key measures	Analytical use
Zone Hourly Demand Profile	Pickup zone × date × hour	Trip count; mean distance and duration	Identify local peak periods.
Airport Transfer Summary	Month × airport × counterpart zone × direction	Trip count; mean distance, duration and total amount	Compare airport access and travel direction.
Weather-Sensitive Zone Demand	Month × pickup zone × weather category	Trip count; observed weather hours; mean hourly demand	Compare demand across weather conditions.
Origin–Destination Flow	Month × directed zone pair	Trip count; mean distance, duration and total amount	Identify busy routes and directional differences.
Fare and Tip Patterns	Month × pickup borough × payment code	Trip count; mean fare, total amount and recorded-tip metrics	Compare recorded payment patterns.
Trip Distance Distribution	Month × pickup borough × distance band	Trip count and share; mean distance, fare and duration	Compare short- and long-trip demand.

4 Optimization strategy

Execution times reported earlier range from 8.578 to 101.717 seconds. Four Spark optimization techniques are applied where each is structurally applicable: partition pruning, broadcast joins, caching, and Adaptive Query Execution (AQE).

4.1 Partition Pruning

Lets Spark skip entire partition directories at planning time when a query filters directly on the partition columns (`pickup_year`, `pickup_month`), instead of scanning every partition and filtering rows afterward.

4.2 Broadcast Join

Copies a small table to every executor instead of shuffling it, so a large table can be joined against it without a network shuffle. Applies where one side of a join is small relative to the other, such as `taxi_zones` against `taxi_trips`.

4.3 Caching

Keeps a computed intermediate result in memory so that later queries reusing the same result skip recomputation. Applies where multiple queries independently compute the same aggregation.

4.4 Adaptive Query Execution

Lets Spark revise a query's physical plan during execution once real intermediate sizes are known, instead of relying only on estimates made before execution starts. Unlike the other three techniques, this is a session-level setting rather than a query rewrite.

Each technique is compared against a baseline on the same query, and the optimized query is verified to produce identical results before any timing conclusion is drawn.

5 Engineering decisions and trade-offs

5.1 Partition Pruning

None of the six analytical queries filter on `pickup_year` or `pickup_month`; each aggregates across the full trip history, so none can demonstrate pruning as written. A month-scoped variant of the Monthly Demand by Zone measure is used instead, the only technique in this study evaluated on a query that does not appear verbatim in the query design above. The trade-off is that pruning does not help the current query library unless month- or quarter-scoped queries are added later.

5.2 Broadcast Join

`taxi_zones` has a fixed 265 rows regardless of trip volume, while `taxi_trips` grows with every month of data ingested; this asymmetry is why the Monthly Demand by Zone join is a broadcast candidate. Broadcasting copies the entire small table to every executor's memory, a cost that is negligible at the current table size but would need reconsideration if a combined multi-city zone lookup table grew large enough to cross Spark's automatic broadcast threshold.

5.3 Caching

The Air Quality and Demand analysis computes the same borough-hour aggregation independently in both the band and correlation queries. Caching this shared intermediate removes the duplication. The trade-off is a staleness risk not shared by the other techniques: a cached result is not automatically refreshed if the underlying table changes, and caching only pays off when a result is reused more than once.

5.4 Adaptive Query Execution

AQE is evaluated by toggling `spark.sql.adaptive.enabled` across several queries rather than by rewriting any query. Its effect on this platform's current workload is discussed in the benchmark report.